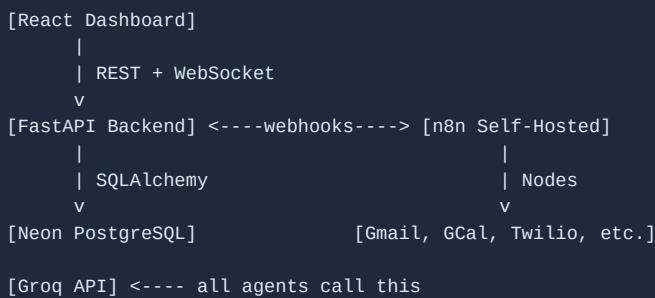


FlowZint — n8n Implementation Guide

All 6 Agents · Webhooks · APIs · DB · Backend

All 6 Agents, Webhooks, APIs, DB, and Backend Integration

1. Architecture Overview



Data Flow for a New Lead

- Lead submits form on website
- FastAPI receives POST /api/leads
- FastAPI saves lead to DB
- FastAPI calls n8n webhook: /webhook/lead-intake
- n8n Workflow 1 starts:
 - Calls FastAPI POST /agents/qualify
 - FastAPI runs Qualifier Agent (Groq)
 - Returns score, intent, mood
 - n8n routes based on intent
- n8n Workflow 2 starts:
 - Calls FastAPI POST /agents/outreach
 - FastAPI writes email via Groq
 - n8n Gmail node sends email
- n8n calls back FastAPI POST /internal/event
- FastAPI broadcasts via WebSocket
- Dashboard updates live

2. Infrastructure Setup

Services Required (All Free)

Service	Purpose	URL
Railway.app	Host FastAPI + n8n	railway.app
Neon.tech	PostgreSQL database	neon.tech
Vercel	Host React frontend	vercel.com
Groq	LLM inference (free tier)	console.groq.com
Gmail API	Send emails	console.cloud.google.com

Google Calendar API	Book meetings	same as above
ElevenLabs	TTS for voice	elevenlabs.io
Clerk	Authentication	clerk.com

Railway Setup for n8n

```
# On Railway, create new service
# Select: Deploy from Docker Image
# Image: n8nio/n8n:latest

# Environment variables to set in Railway:
N8N_HOST=0.0.0.0
N8N_PORT=5678
N8N_PROTOCOL=https
WEBHOOK_URL=https://your-n8n-railway-url.railway.app
N8N_ENCRYPTION_KEY=generate_32_char_random_string
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=your_strong_password
DB_TYPE=sqlite
N8N_USER_MANAGEMENT_DISABLED=false

# Volume: mount /home/node/.n8n for persistence
```

FastAPI on Railway

```
# Create Dockerfile in backend folder
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

3. Database Schema

SQL — Run on Neon.tech

```

-- Create all tables

CREATE TABLE leads (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) NOT NULL,
  email       VARCHAR(255),
  phone       VARCHAR(50),
  company     VARCHAR(255),
  message     TEXT,
  source      VARCHAR(100) DEFAULT 'website',
  stage       VARCHAR(50)  DEFAULT 'new',
  score       INTEGER      DEFAULT 0,
  intent      VARCHAR(50)  DEFAULT 'unknown',
  mood        VARCHAR(50)  DEFAULT 'neutral',
  buying_stage VARCHAR(50)  DEFAULT 'awareness',
  conversion_probability FLOAT DEFAULT 0.0,
  predicted_objection TEXT,
  recommended_action TEXT,
  assigned_to VARCHAR(255),
  created_at  TIMESTAMP    DEFAULT NOW(),
  updated_at  TIMESTAMP    DEFAULT NOW()
);

CREATE TABLE interactions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id     UUID REFERENCES leads(id) ON DELETE CASCADE,
  type        VARCHAR(50)  NOT NULL,
  direction   VARCHAR(20)  DEFAULT 'outbound',
  channel     VARCHAR(50)  DEFAULT 'email',
  subject     TEXT,
  content     TEXT,
  sentiment   VARCHAR(50),
  agent_used  VARCHAR(100),
  n8n_execution_id VARCHAR(255),
  created_at  TIMESTAMP    DEFAULT NOW()
);

CREATE TABLE agent_events (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id     UUID REFERENCES leads(id) ON DELETE CASCADE,
  agent_name  VARCHAR(100) NOT NULL,
  event_type  VARCHAR(50)  NOT NULL,
  input_data  JSONB,
  output_data JSONB,
  reasoning   TEXT,
  duration_ms INTEGER,
  created_at  TIMESTAMP    DEFAULT NOW()
);

CREATE TABLE workflow_executions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id     UUID REFERENCES leads(id),
  workflow_name VARCHAR(100) NOT NULL,
  n8n_execution_id VARCHAR(255),
  status      VARCHAR(50)  DEFAULT 'running',
  current_step VARCHAR(100),
  steps_completed JSONB    DEFAULT '[]',
  started_at  TIMESTAMP    DEFAULT NOW(),
  completed_at TIMESTAMP
);

```

```

CREATE TABLE meetings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id     UUID REFERENCES leads(id),
  title       VARCHAR(255),
  scheduled_at TIMESTAMP,
  duration_mins INTEGER      DEFAULT 30,
  google_event_id VARCHAR(255),
  meet_link   VARCHAR(500),
  status      VARCHAR(50)   DEFAULT 'scheduled',
  created_at  TIMESTAMP     DEFAULT NOW()
);

CREATE TABLE outreach_sequences (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id     UUID REFERENCES leads(id),
  sequence_step INTEGER      DEFAULT 1,
  channel     VARCHAR(50),
  subject     TEXT,
  content     TEXT,
  scheduled_at TIMESTAMP,
  sent_at     TIMESTAMP,
  opened_at   TIMESTAMP,
  replied_at  TIMESTAMP,
  status      VARCHAR(50)   DEFAULT 'pending',
  created_at  TIMESTAMP     DEFAULT NOW()
);

-- Indexes for performance
CREATE INDEX idx_leads_stage    ON leads(stage);

```

```

CREATE INDEX idx_leads_intent   ON leads(intent);
CREATE INDEX idx_interactions_lead ON interactions(lead_id);
CREATE INDEX idx_agent_events_lead ON agent_events(lead_id);
CREATE INDEX idx_workflow_lead  ON workflow_executions(lead_id);

```

SQLAlchemy Models — `backend/core/database.py`

```

from sqlalchemy import create_engine, Column, String, Integer, \
    Float, Text, DateTime, ForeignKey, JSON
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, relationship
from sqlalchemy.dialects.postgresql import UUID
import uuid
import os
from datetime import datetime

DATABASE_URL = os.getenv("DATABASE_URL")
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class Lead(Base):
    __tablename__ = "leads"
    id = Column(UUID(as_uuid=True), primary_key=True,
        default=uuid.uuid4)
    name = Column(String(255), nullable=False)
    email = Column(String(255))
    phone = Column(String(50))

```

```

company          = Column(String(255))
message          = Column(Text)
source           = Column(String(100), default="website")
stage            = Column(String(50), default="new")
score            = Column(Integer, default=0)
intent           = Column(String(50), default="unknown")
mood             = Column(String(50), default="neutral")
buying_stage     = Column(String(50), default="awareness")
conversion_probability = Column(Float, default=0.0)
predicted_objection = Column(Text)
recommended_action = Column(Text)
created_at       = Column(DateTime, default=datetime.utcnow)
updated_at       = Column(DateTime, default=datetime.utcnow,
                        onupdate=datetime.utcnow)
interactions     = relationship("Interaction", back_populates="lead")
agent_events     = relationship("AgentEvent", back_populates="lead")

```

```

class Interaction(Base):
    __tablename__ = "interactions"
    id             = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    lead_id        = Column(UUID(as_uuid=True), ForeignKey("leads.id"))
    type           = Column(String(50))
    direction      = Column(String(20), default="outbound")
    channel         = Column(String(50), default="email")
    subject        = Column(Text)
    content        = Column(Text)
    sentiment      = Column(String(50))
    agent_used     = Column(String(100))
    n8n_execution_id = Column(String(255))

```

```

created_at       = Column(DateTime, default=datetime.utcnow)
lead             = relationship("Lead", back_populates="interactions")

```

```

class AgentEvent(Base):
    __tablename__ = "agent_events"
    id             = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    lead_id        = Column(UUID(as_uuid=True), ForeignKey("leads.id"))
    agent_name     = Column(String(100))
    event_type     = Column(String(50))
    input_data     = Column(JSON)
    output_data    = Column(JSON)
    reasoning      = Column(Text)
    duration_ms    = Column(Integer)
    created_at     = Column(DateTime, default=datetime.utcnow)
    lead           = relationship("Lead", back_populates="agent_events")

```

```

class WorkflowExecution(Base):
    __tablename__ = "workflow_executions"
    id             = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    lead_id        = Column(UUID(as_uuid=True), ForeignKey("leads.id"))
    workflow_name  = Column(String(100))
    n8n_execution_id = Column(String(255))
    status         = Column(String(50), default="running")
    current_step   = Column(String(100))
    steps_completed = Column(JSON, default=list)
    started_at     = Column(DateTime, default=datetime.utcnow)
    completed_at   = Column(DateTime)

```

```
class Meeting(Base):
    __tablename__ = "meetings"
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    lead_id = Column(UUID(as_uuid=True), ForeignKey("leads.id"))
    title = Column(String(255))
    scheduled_at = Column(DateTime)
    duration_mins = Column(Integer, default=30)
    google_event_id = Column(String(255))
    meet_link = Column(String(500))
    status = Column(String(50), default="scheduled")
    created_at = Column(DateTime, default=datetime.utcnow)
```

4. Environment Variables

`backend/.env`

```
# Database
DATABASE_URL=postgresql://user:pass@ep-xxx.neon.tech/flowzint?sslmode=require

# Groq (free LLM)
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
GROQ_MODEL=llama-3.3-70b-versatile

# n8n connection
N8N_BASE_URL=https://your-n8n.railway.app
N8N_API_KEY=your_n8n_api_key_from_settings
N8N_WEBHOOK_BASE=https://your-n8n.railway.app/webhook

# Gmail / Google
GMAIL_CLIENT_ID=xxx.apps.googleusercontent.com
GMAIL_CLIENT_SECRET=GOCSPX-xxx
GMAIL_REFRESH_TOKEN=1/xxx
GMAIL_SENDER=youremail@gmail.com
GOOGLE_CALENDAR_ID=primary

# ElevenLabs TTS
ELEVENLABS_API_KEY=sk_xxx
ELEVENLABS_VOICE_ID=21m00Tcm4TlvDq8ikWAM

# Clerk
CLERK_SECRET_KEY=sk_test_xxx

# App
FRONTEND_URL=https://flowzint.vercel.app
APP_SECRET=random_32_char_string
```

`frontend/.env`

```
VITE_API_URL=https://your-backend.railway.app
VITE_WS_URL=wss://your-backend.railway.app
VITE_CLERK_PUBLISHABLE_KEY=pk_test_xxx
```

5. FastAPI Backend — All Endpoints

`backend/main.py`

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from core.database import engine, Base
from routers import leads, agents, voice, ws, internal
import os

Base.metadata.create_all(bind=engine)

app = FastAPI(title="FlowZint API", version="1.0.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=[os.getenv("FRONTEND_URL", "")],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(leads.router, prefix="/api")
app.include_router(agents.router, prefix="/agents")
app.include_router(voice.router, prefix="/voice")
app.include_router(ws.router)
app.include_router(internal.router, prefix="/internal")

@app.get("/health")
def health():
    return {"status": "ok", "service": "flowzint-backend"}

```

``backend/routers/leads.py``

```

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from core.database import get_db, Lead
from pydantic import BaseModel
from typing import Optional
import httpx, os

router = APIRouter(tags=["leads"])

class LeadCreate(BaseModel):
    name: str
    email: Optional[str]
    phone: Optional[str]
    company: Optional[str]
    message: str
    source: Optional[str] = "website"

class LeadStageUpdate(BaseModel):
    stage: str

@router.post("/leads")
async def create_lead(payload: LeadCreate, db: Session = Depends(get_db)):
    lead = Lead(**payload.dict())
    db.add(lead)
    db.commit()
    db.refresh(lead)

    # Trigger n8n workflow immediately
    await trigger_n8n_lead_intake(lead)
    return lead

```

```

@router.get("/leads")
def get_leads(db: Session = Depends(get_db)):
    return db.query(Lead).order_by(Lead.created_at.desc()).all()

@router.get("/leads/{lead_id}")
def get_lead(lead_id: str, db: Session = Depends(get_db)):
    lead = db.query(Lead).filter(Lead.id == lead_id).first()
    if not lead:
        raise HTTPException(status_code=404, detail="Lead not found")
    return lead

@router.patch("/leads/{lead_id}/stage")
def update_stage(lead_id: str, payload: LeadStageUpdate,
                 db: Session = Depends(get_db)):
    lead = db.query(Lead).filter(Lead.id == lead_id).first()
    if not lead:
        raise HTTPException(status_code=404, detail="Lead not found")
    lead.stage = payload.stage
    db.commit()
    return lead

@router.get("/leads/{lead_id}/interactions")
def get_interactions(lead_id: str, db: Session = Depends(get_db)):
    lead = db.query(Lead).filter(Lead.id == lead_id).first()
    if not lead:
        raise HTTPException(status_code=404, detail="Lead not found")
    return lead.interactions

async def trigger_n8n_lead_intake(lead):

```

```

    webhook_url = f"{os.getenv('N8N_WEBHOOK_BASE')}/lead-intake"
    async with httpx.AsyncClient() as client:
        try:
            await client.post(webhook_url, json={
                "lead_id": str(lead.id),
                "name": lead.name,
                "email": lead.email,
                "phone": lead.phone,
                "company": lead.company,
                "message": lead.message,
                "source": lead.source,
            }, timeout=10.0)
        except Exception as e:
            print(f"[n8n trigger failed] {e}")

```

``backend/routers/agents.py`` — All 6 Agent Endpoints

[illegible]

[illegible]

[illegible]

backend/routers/internal.py — n8n Callback Receiver

```

from fastapi import APIRouter, Request
from core.ws_manager import ws_manager, EventType
from core.database import get_db, WorkflowExecution, Lead
from sqlalchemy.orm import Session
from fastapi import Depends

router = APIRouter(tags=["internal"])

# n8n calls this after every workflow step
@router.post("/internal/event")
async def workflow_event(request: Request, db: Session = Depends(get_db)):
    body = await request.json()

    lead_id = body.get("lead_id")
    event = body.get("event") # e.g. "outreach_sent"
    step = body.get("step") # e.g. "email_sent"
    workflow_name = body.get("workflow_name")
    n8n_execution_id = body.get("execution_id")
    data = body.get("data", {})

    # Update workflow execution in DB
    execution = db.query(WorkflowExecution).filter(
        WorkflowExecution.n8n_execution_id == n8n_execution_id
    ).first()

    if execution:
        execution.current_step = step
        steps = execution.steps_completed or []
        steps.append({"step": step, "event": event})
        execution.steps_completed = steps

```

```

        db.commit()

    # Broadcast to React dashboard via WebSocket
    await ws_manager.broadcast(EventType.WORKFLOW_STEP, {
        "lead_id":      lead_id,
        "event":        event,
        "step":         step,
        "workflow_name": workflow_name,
        "data":         data,
    })

    return {"status": "ok"}

# n8n calls this when a full workflow completes
@router.post("/internal/workflow-complete")
async def workflow_complete(request: Request, db: Session = Depends(get_db)):
    body = await request.json()

    lead_id      = body.get("lead_id")
    workflow_name = body.get("workflow_name")
    final_stage   = body.get("final_stage")

    if final_stage:
        lead = db.query(Lead).filter(Lead.id == lead_id).first()
        if lead:
            lead.stage = final_stage
            db.commit()

    await ws_manager.broadcast(EventType.WORKFLOW_COMPLETE, {
        "lead_id":      lead_id,

        "workflow_name": workflow_name,
        "final_stage":   final_stage,
    })

    return {"status": "ok"}

```

6. n8n Setup on Railway

Step 1 — Access n8n

After Railway deploys, open your n8n URL. Login with the credentials you set.

Step 2 — Enable API

```

Settings → API → Enable API → Copy the API key
→ Save this as N8N_API_KEY in your FastAPI .env

```

Step 3 — Set Variables in n8n

```

Settings → Variables → Add:

FASTAPI_URL      = https://your-backend.railway.app
FRONTEND_URL     = https://flowzint.vercel.app

```

7. n8n Credentials to Create

Go to: **Credentials** → **New** in n8n

Credential 1 — Gmail OAuth2

```
Type: Gmail OAuth2 API
Client ID:      from Google Cloud Console
Client Secret:  from Google Cloud Console
Scopes:         https://www.googleapis.com/auth/gmail.send
Click: Connect my account → Login with Google
```

Credential 2 — Google Calendar OAuth2

```
Type: Google Calendar OAuth2 API
Client ID:      same as Gmail
Client Secret:  same as Gmail
Scopes:         https://www.googleapis.com/auth/calendar
```

Credential 3 — HTTP Header Auth (for FastAPI)

```
Type: Header Auth
Name: Authorization
Value: Bearer your_app_secret_from_env
```

8. Workflow 1 — Lead Intake & Qualifier Agent

In n8n UI: New Workflow → Name: "01 Lead Intake + Qualifier"

Node 1: Webhook (Entry Point)

```
Type:      Webhook
HTTP Method: POST
Path:      lead-intake
Auth:      None (FastAPI already validated)
Response Mode: Respond immediately

→ This creates URL: https://your-n8n.railway.app/webhook/lead-intake
→ Save this URL as N8N_WEBHOOK_BASE in FastAPI .env
```

Node 2: Create Workflow Execution Record

```
Type: HTTP Request
Name: Create Execution Record
Method: POST
URL:    {{ $vars.FASTAPI_URL }}/internal/event
Auth:   Header Auth credential
Body (JSON):
{
  "lead_id":    "{{ $json.lead_id }}",
  "event":      "pipeline_started",
  "step":       "intake",
  "workflow_name": "lead_intake",
  "execution_id": "{{ $execution.id }}"
}
```

Node 3: Call Qualifier Agent

```
Type: HTTP Request
Name: Qualify Lead
Method: POST
URL: {{ $vars.FASTAPI_URL }}/agents/qualify
Auth: Header Auth credential
Body (JSON):
{
  "lead_id": "{{ $('Webhook').item.json.lead_id }}",
  "name": "{{ $('Webhook').item.json.name }}",
  "message": "{{ $('Webhook').item.json.message }}",
  "company": "{{ $('Webhook').item.json.company }}",
  "source": "{{ $('Webhook').item.json.source }}"
}
```

Node 4: Switch — Route by Intent

```
Type: Switch
Name: Route by Intent
Mode: Rules
Value: {{ $json.intent }}

Rule 1: equals "high" → Output 0 (High Intent)
Rule 2: equals "medium" → Output 1 (Medium Intent)
Rule 3: equals "low" → Output 2 (Low Intent)
Fallback Output: 1
```

Node 5a (High Intent): Notify Dashboard → Trigger Outreach Workflow

```
Type: HTTP Request
Name: Notify High Intent
Method: POST
URL: {{ $vars.FASTAPI_URL }}/internal/event
Body:
{
  "lead_id": "{{ $('Webhook').item.json.lead_id }}",
  "event": "lead_qualified_high",
  "step": "qualification_complete",
  "workflow_name": "lead_intake",
  "execution_id": "{{ $execution.id }}",
  "data": {
    "score": "{{ $('Qualify Lead').item.json.score }}",
    "intent": "high",
    "mood": "{{ $('Qualify Lead').item.json.mood }}"
  }
}

→ Connect to: Trigger Outreach Workflow node
```

Node 5b (Medium Intent): Wait 1 hour then trigger outreach

```
Type: Wait
Name: Wait 1 Hour
Wait: 1 hour

→ Connect to: Trigger Outreach Workflow node
```

Node 5c (Low Intent): Add to nurture sequence


```

Type: HTTP Request
Name: Notify Low Intent
Method: POST
URL: {{ $vars.FASTAPI_URL }}/internal/event
Body:
{
  "lead_id": "{{ $('Webhook').item.json.lead_id }}",
  "event": "lead_low_intent",
  "step": "added_to_nurture",
  "workflow_name": "lead_intake",
  "execution_id": "{{ $execution.id }}"
}

```

Node 6: Trigger Outreach Workflow

```

Type: HTTP Request
Name: Trigger Outreach
Method: POST
URL: https://your-n8n.railway.app/webhook/outreach-start
Body:
{
  "lead_id": "{{ $('Webhook').item.json.lead_id }}",
  "name": "{{ $('Webhook').item.json.name }}",
  "email": "{{ $('Webhook').item.json.email }}",
  "company": "{{ $('Webhook').item.json.company }}",
  "intent": "{{ $('Qualify Lead').item.json.intent }}",
  "mood": "{{ $('Qualify Lead').item.json.mood }}",
  "score": "{{ $('Qualify Lead').item.json.score }}",
  "predicted_objection": "{{ $('Qualify Lead').item.json.predicted_objection }}"
}

```

→ **Activate this workflow**

9. Workflow 2 — Outreach Agent

New Workflow → Name: "02 Outreach Agent"

Node 1: Webhook

```

Type: Webhook
Method: POST
Path: outreach-start

```

Node 2: Call Outreach Agent

```

Type: HTTP Request
Name: Generate Outreach
Method: POST
URL: {{ $vars.FASTAPI_URL }}/agents/outreach
Body:
{
  "lead_id": "{{ $json.lead_id }}",
  "name": "{{ $json.name }}",
  "company": "{{ $json.company }}",
  "intent": "{{ $json.intent }}",
  "mood": "{{ $json.mood }}",
  "score": "{{ $json.score }}",
  "predicted_objection": "{{ $json.predicted_objection }}"
}

```

Node 3: Send Email via Gmail

```
Type:      Gmail
Operation: Send
To:        {{ $('Webhook').item.json.email }}
Subject:   {{ $('Generate Outreach').item.json.email_subject }}
Message:   {{ $('Generate Outreach').item.json.email_body }}
Type:      HTML
```

Node 4: Notify Dashboard

```
Type:      HTTP Request
Name:      Notify Email Sent
Method:    POST
URL:       {{ $vars.FASTAPI_URL }}/internal/event
Body:      {
  "lead_id":  "{{ $('Webhook').item.json.lead_id }}",
  "event":    "outreach_sent",
  "step":     "email_sent",
  "workflow_name": "outreach",
  "execution_id": "{{ $execution.id }}",
  "data": {
    "subject": "{{ $('Generate Outreach').item.json.email_subject }}",
    "channel": "email"
  }
}
```

Node 5: Wait for Reply — 48 hours

```
Type:      Wait
Name:      Wait for Reply
Mode:      Wait for Webhook Call
Path:      reply-received/{{ $('Webhook').item.json.lead_id }}
Timeout:   48 hours

On Timeout → Connect to: Trigger Follow-up Workflow
On Reply   → Connect to: Analyze Reply
```

Node 6: Analyze Reply Sentiment

```
Type:      HTTP Request
Name:      Analyze Sentiment
Method:    POST
URL:       {{ $vars.FASTAPI_URL }}/agents/negotiate
Body:      {
  "lead_id":      "{{ $('Webhook').item.json.lead_id }}",
  "customer_message": "{{ $json.reply_text }}",
  "history":      [],
  "intent":       "{{ $('Webhook').item.json.intent }}"
}
```

Node 7: Switch — Route by Reply Type

```
Type:      Switch
Value:     {{ $json.detected_objection_type }}

Rule 1: equals "price"      → Output 0
Rule 2: equals "timing"     → Output 1
Rule 3: equals "trust"     → Output 2
Rule 4: equals "interested" → Output 3 (→ Trigger Scheduler)
Fallback: Output 2
```

Node 8 (interested): Trigger Scheduler Workflow

```
Type: HTTP Request
Name: Trigger Scheduler
Method: POST
URL: https://your-n8n.railway.app/webhook/schedule-meeting
Body:
{
  "lead_id": "{{ $('Webhook').item.json.lead_id }}",
  "name": "{{ $('Webhook').item.json.name }}",
  "email": "{{ $('Webhook').item.json.email }}",
  "timezone": "Asia/Kolkata"
}
```

Node 8 (objection): Trigger Negotiation Workflow

```
Type: HTTP Request
Name: Trigger Negotiation
Method: POST
URL: https://your-n8n.railway.app/webhook/negotiate
Body:
{
  "lead_id": "{{ $('Webhook').item.json.lead_id }}",
  "name": "{{ $('Webhook').item.json.name }}",
  "email": "{{ $('Webhook').item.json.email }}",
  "objection_type": "{{ $json.detected_objection_type }}",
  "customer_message": "{{ $json.customer_message }}",
  "response_message": "{{ $json.response_message }}"
}
```

→ **Activate this workflow**

10. Workflow 3 — Negotiation Agent

New Workflow → Name: "03 Negotiation Agent"

Node 1: Webhook

```
Type: Webhook
Method: POST
Path: negotiate
```

Node 2: Call Negotiation Agent

```
Type: HTTP Request
Name: Handle Objection
Method: POST
URL: {{ $vars.FASTAPI_URL }}/agents/negotiate
Body:
{
  "lead_id": "{{ $json.lead_id }}",
  "customer_message": "{{ $json.customer_message }}",
  "history": [],
  "intent": "medium"
}
```

Node 3: Send Negotiation Response Email

```
Type: Gmail
Operation: Send
To: {{ $('Webhook').item.json.email }}
Subject: Re: {{ $('Webhook').item.json.objection_type }} – Following up
Message: {{ $('Handle Objection').item.json.response_message }}
```

Node 4: Notify Dashboard

```

Type: HTTP Request
Name: Notify Negotiation
Method: POST
URL: {{ $vars.FASTAPI_URL }}/internal/event
Body:
{
  "lead_id": "{{ $('Webhook').item.json.lead_id }}",
  "event": "negotiation_response_sent",
  "step": "objection_handled",
  "workflow_name": "negotiation",
  "execution_id": "{{ $execution.id }}",
  "data": {
    "objection_type": "{{ $('Webhook').item.json.objection_type }}",
    "strategy": "{{ $('Handle Objection').item.json.strategy_used }}",
    "conversion_after": "{{ $('Handle Objection').item.json.conversion_probability }}"
  }
}

```

Node 5: Wait for Second Reply — 24 hours

```

Type: Wait
Mode: Timeout
Timeout: 24 hours

On Timeout → Trigger Follow-up
On Resume → Check if interested → Trigger Scheduler

```

→ **Activate this workflow**

11. Workflow 4 — Follow-up Agent (Cron + Triggered)

New Workflow → Name: "04 Follow-up Agent"

Node 1A: Schedule Trigger (Cron — runs every day 10am)

```

Type: Schedule Trigger
Interval: Every Day
Time: 10:00 AM

```

Node 1B: Webhook Trigger (triggered by other workflows)

```

Type: Webhook
Method: POST
Path: followup-start

```

Node 2 (from Cron): Fetch Stale Leads from DB

```

Type: HTTP Request
Name: Get Stale Leads
Method: GET
URL: {{ $vars.FASTAPI_URL }}/api/leads/stale
      (FastAPI returns leads with no interaction > 3 days)

```

Node 3: Loop Over Each Lead

```

Type: Split In Batches
Batch Size: 1

```

Node 4: Call Follow-up Agent

```

Type: HTTP Request
Name: Generate Follow-up
Method: POST
URL: {{ $vars.FASTAPI_URL }}/agents/followup
Body:
{
  "lead_id": "{{ $json.id }}",
  "name": "{{ $json.name }}",
  "company": "{{ $json.company }}",
  "last_contact": "{{ $json.updated_at }}",
  "days_since": "{{ $json.days_since_contact }}",
  "previous_channel": "email"
}

```

Node 5: Send Follow-up Email

```

Type: Gmail
Operation: Send
To: {{ $json.email }}
Subject: {{ $('Generate Follow-up').item.json.subject }}
Message: {{ $('Generate Follow-up').item.json.message }}

```

Node 6: Notify Dashboard

```

Type: HTTP Request
Method: POST
URL: {{ $vars.FASTAPI_URL }}/internal/event
Body:
{
  "lead_id": "{{ $json.id }}",
  "event": "followup_sent",
  "step": "followup_email_sent",
  "workflow_name": "followup",
  "execution_id": "{{ $execution.id }}"
}

```

→ **Activate this workflow**

12. Workflow 5 — Scheduler Agent

New Workflow → Name: "05 Scheduler Agent"

Node 1: Webhook

```

Type: Webhook
Method: POST
Path: schedule-meeting

```

Node 2: Call Scheduler Agent

```

Type: HTTP Request
Name: Find Best Time
Method: POST
URL: {{ $vars.FASTAPI_URL }}/agents/schedule
Body:
{
  "lead_id": "{{ $json.lead_id }}",
  "name": "{{ $json.name }}",
  "email": "{{ $json.email }}",
  "preferred_time": "{{ $json.preferred_time }}",
  "timezone": "{{ $json.timezone }}"
}

```

Node 3: Create Google Calendar Event

```
Type:      Google Calendar
Operation: Create Event
Calendar:  Primary
Title:     FlowZint Demo — {{ $('Webhook').item.json.name }}
Start:     {{ $('Find Best Time').item.json.meeting_start }}
End:       {{ $('Find Best Time').item.json.meeting_end }}
Description: AI-scheduled demo call
Attendees: {{ $('Webhook').item.json.email }}
Add Meet Link: true
```

Node 4: Send Confirmation Email

```
Type:      Gmail
Operation: Send
To:        {{ $('Webhook').item.json.email }}
Subject:   Meeting Confirmed — FlowZint Demo
Message:
Hi {{ $('Webhook').item.json.name }},

Your demo is confirmed!

Date: {{ $('Find Best Time').item.json.meeting_date }}
Time: {{ $('Find Best Time').item.json.meeting_time }}
Link: {{ $('Create Calendar Event').item.json.hangoutLink }}

Looking forward to it!

Best,
FlowZint Team
```

Node 5: Notify Dashboard

```
Type:      HTTP Request
Method:    POST
URL:       {{ $vars.FASTAPI_URL }}/internal/event
Body:
{
  "lead_id":  "{{ $('Webhook').item.json.lead_id }}",
  "event":    "meeting_booked",
  "step":     "calendar_event_created",
  "workflow_name": "scheduler",
  "execution_id": "{{ $execution.id }}",
  "data": {
    "meeting_time": "{{ $('Find Best Time').item.json.meeting_start }}",
    "meet_link":    "{{ $('Create Calendar Event').item.json.hangoutLink }}"
  }
}
```

→ **Activate this workflow**

13. Workflow 6 — Voice Agent

Voice is WebSocket-based (not n8n triggered)

n8n only handles post-call follow-through

New Workflow → Name: "06 Voice Post-Call"

Node 1: Webhook

```
Type:      Webhook
Method:    POST
Path:      voice-call-complete
(FastAPI calls this after voice call ends)
```

Node 2: Switch on Call Outcome

```
Type: Switch
Value: {{ $json.outcome }}

Rule 1: "meeting_requested" → Trigger Scheduler
Rule 2: "follow_up_needed" → Wait 2hrs → Outreach
Rule 3: "not_interested" → Update stage → closed_lost
```

Node 3: Trigger Scheduler (if meeting requested)

```
Same as Workflow 5 Node 1 trigger
URL: https://your-n8n.railway.app/webhook/schedule-meeting
Body:
{
  "lead_id": "{{ $json.lead_id }}",
  "name":    "{{ $json.name }}",
  "email":   "{{ $json.email }}"
}
```

Node 4: Notify Dashboard

```
Type: HTTP Request
Method: POST
URL:   {{ $vars.FASTAPI_URL }}/internal/event
Body:
{
  "lead_id": "{{ $json.lead_id }}",
  "event":   "voice_call_completed",
  "step":    "{{ $json.outcome }}",
  "workflow_name": "voice",
  "execution_id": "{{ $execution.id }}",
  "data": {
    "duration_secs": "{{ $json.duration_secs }}",
    "transcript_summary": "{{ $json.summary }}"
  }
}
```

→ **Activate this workflow**

14. All Webhook URLs Summary

After activating all workflows, you will have these webhook URLs. Save all of them.

```
Workflow 1 – Lead Intake:
https://your-n8n.railway.app/webhook/lead-intake

Workflow 2 – Outreach:
https://your-n8n.railway.app/webhook/outreach-start

Workflow 2 – Reply received (dynamic):
https://your-n8n.railway.app/webhook/reply-received/{lead_id}

Workflow 3 – Negotiation:
https://your-n8n.railway.app/webhook/negotiate

Workflow 4 – Follow-up:
https://your-n8n.railway.app/webhook/followup-start

Workflow 5 – Schedule meeting:
https://your-n8n.railway.app/webhook/schedule-meeting

Workflow 6 – Voice complete:
https://your-n8n.railway.app/webhook/voice-call-complete
```

All of these go into your FastAPI `.env` as:

```
N8N_WEBHOOK_LEAD_INTAKE=https://your-n8n.railway.app/webhook/lead-intake
N8N_WEBHOOK_OUTREACH=https://your-n8n.railway.app/webhook/outreach-start
N8N_WEBHOOK_NEGOTIATE=https://your-n8n.railway.app/webhook/negotiate
N8N_WEBHOOK_FOLLOWUP=https://your-n8n.railway.app/webhook/followup-start
N8N_WEBHOOK_SCHEDULE=https://your-n8n.railway.app/webhook/schedule-meeting
N8N_WEBHOOK_VOICE=https://your-n8n.railway.app/webhook/voice-call-complete
```

15. WebSocket Event Reference

Every agent and workflow step broadcasts one of these events via WebSocket to the React dashboard.

```
class EventType(str, Enum):
    AGENT_THINKING      = "agent_thinking"
    AGENT_DECISION      = "agent_decision"
    WORKFLOW_STEP      = "workflow_step"
    WORKFLOW_COMPLETE  = "workflow_complete"
    LEAD_CREATED        = "lead_created"
    LEAD_SCORED         = "lead_scored"
    LEAD_STAGE_CHANGED = "lead_stage_changed"
    EMAIL_SENT          = "email_sent"
    MEETING_BOOKED      = "meeting_booked"
    CALL_STARTED        = "call_started"
    CALL_ENDED          = "call_ended"
    FOLLOWUP_SENT        = "followup_sent"
    NEGOTIATION_STARTED = "negotiation_started"
```

Example WebSocket message format

```
{
  "event": "agent_decision",
  "timestamp": 1716289200.0,
  "data": {
    "lead_id": "uuid-here",
    "agent": "Qualifier Agent",
    "result": {
      "score": 87,
      "intent": "high",
      "mood": "interested",
      "conversion_probability": 0.87,
      "reasoning": "High urgency language, B2B SaaS company, clear pain point"
    },
    "duration_ms": 1240
  }
}
```

16. Testing Each Workflow

Test Workflow 1

```
curl -X POST https://your-backend.railway.app/api/leads \
-H "Content-Type: application/json" \
-d '{
  "name": "Test Lead",
  "email": "test@example.com",
  "company": "Acme Corp",
  "message": "We want to automate our sales team of 20 people",
  "source": "website"
}'

# Expected: Lead created → n8n webhook fires →
#           Qualifier agent runs → Dashboard updates
```


Test Workflow 2

```
# After Workflow 1 runs, Workflow 2 auto-triggers
# Check n8n execution logs in n8n UI → Executions
# Check Gmail for sent email
# Check FastAPI logs for /agents/outreach call
```

Test Reply (Workflow 3)

```
# Simulate a reply coming in (normally from Gmail webhook)
curl -X POST https://your-n8n.railway.app/webhook/reply-received/{lead_id} \
  -H "Content-Type: application/json" \
  -d '{"reply_text": "This looks interesting but the price seems high"}'
```

Test Follow-up (Workflow 4)

```
# Add stale leads endpoint to FastAPI
# GET /api/leads/stale returns leads with no contact > 3 days
# Then trigger manually:
curl -X POST https://your-n8n.railway.app/webhook/followup-start \
  -d '{"lead_id": "uuid", "name": "Test", "days_since": 4}'
```

Test Scheduler (Workflow 5)

```
curl -X POST https://your-n8n.railway.app/webhook/schedule-meeting \
  -H "Content-Type: application/json" \
  -d '{
    "lead_id": "uuid",
    "name": "Test Lead",
    "email": "test@example.com",
    "timezone": "Asia/Kolkata"
  }'
# Expected: Google Calendar event created → confirmation email sent
```

17. Deployment Checklist

Before Demo

```
Backend (Railway):
[ ] FastAPI deployed and /health returns 200
[ ] DATABASE_URL connected to Neon
[ ] GROQ_API_KEY working (test /agents/qualify)
[ ] Gmail credentials connected
[ ] Google Calendar credentials connected
[ ] All 6 agent endpoints return valid JSON
[ ] WebSocket /ws/dashboard connects from frontend

n8n (Railway):
[ ] n8n accessible at your Railway URL
[ ] API key generated and saved
[ ] All 5 workflows created and ACTIVATED
[ ] Gmail credential connected in n8n
[ ] Google Calendar credential connected in n8n
[ ] Variables: FASTAPI_URL set
[ ] Test execution runs without errors

Frontend (Vercel):
[ ] VITE_API_URL points to Railway backend
[ ] VITE_WS_URL points to Railway backend (wss://)
[ ] Clerk auth working
[ ] WebSocket connects and shows live events
[ ] Lead form submits and triggers full pipeline

End-to-End Test:
[ ] Submit test lead → see it in dashboard
[ ] Watch AI Brain Panel update with agent reasoning
[ ] Watch Kanban card move through stages
[ ] Check Gmail for sent outreach email

[ ] Simulate reply → see negotiation agent activate
[ ] Book meeting → see calendar event created
```